

# LTLf synthesis under external knowledge

The prototype, and early results

---

Nadav Alon

12 August 2026

## Where this is

A **prototype** that runs end to end, and a first sweep of measurements.

- Five synthesis methods behind one interface, all producing controllers, not just verdicts.
- Two command-line tools; controllers are machine-checkable.
- A parametric benchmark suite, raced against `ltlfsynt` where the comparison is legitimate.

Today: what ships, what it looks like to use, and what the first numbers say. **Early** numbers — four synthetic families,  $n \leq 8$ .

## The tools

---

## Five methods, one interface

Every method takes the same input — goal formula  $\varphi$ , a variable partition  $(I_f, I_k, O_f, O_k)$ , and a knowledge transducer  $\mathcal{T}_{in} / \mathcal{T}_{out}$  per known set — and is interchangeable at the flag level.

| Method          | Representation       | Built on |
|-----------------|----------------------|----------|
| DfaProduct      | explicit DFA         | Spot     |
| NfaProduct      | explicit NFA         | MONA     |
| MtdfaProduct    | symbolic (MTDFA)     | Spot     |
| MtnfaProduct    | symbolic (MTNFA)     | MONA     |
| OtfMtdfaProduct | symbolic, on-the-fly | Spot     |

Eight method×Representation flags exist; these five are wired, three report *not yet implemented*.

That interchangeability is what makes act 3 possible at all: the methods are subjects of one benchmark, not five separate programs

## Two binaries

`ltlf-ek-synth` — consumes knowledge, emits a verdict or a controller.

```
--dfa-product | --nfa-product | --mtdfa-product | --mtnfa-product | --otf-mtdfa-product
--formula=STRING          the goal formula
--part-file PATH           the variable partition (or --inputs/--outputs shorthand)
--known-input-transducer T_in --known-output-transducer T_out
--realizable               verdict only          --model-check verify a controller
--benchmark PATH           per-stage timing report as JSON
```

Exit codes carry the verdict: 0 realizable, 20 unrealizable, 2 usage.

`ltlf-ek-deps` — *produces* knowledge instead of consuming it.

```
--formula=STRING          --direction in|out
--transducer PATH          write the resulting T_in / T_out
--emit-part PATH           write the updated part file
```

Finds a dependent input set  $X_{dep}$  or dependent output set  $Y_{dep}$ , and can emit it as a ready-to-use transducer.

## Using it

---

# One instance, in full

## The partition — a part file:

```
input_free:  a
input_known: k
output_free: o
output_known:
```

## The goal: $\varphi = X[!] \quad k$

“at the next step,  $k$  holds” — a property about a *known* variable, whose value the controller does not choose.

## The knowledge — $\mathcal{T}_{in}$ as HOA plus a labelling:

```
States: 2    AP: 2 "a" "k"
--BODY--
State: 0
    [t] 1
State: 1
    [t] 1
--END--
%%LAMBDA
state 0: !k
state 1: k
```

$k$  is false initially, true from step 1 onward — regardless of what the environment plays.

# The knowledge is load-bearing

Same  $\varphi$ , same partition, same method. Only  $\mathcal{T}_{in}$  changes.

$\mathcal{T}_{in} = \text{tin-eventual-k}$ :  $k$  turns on at step 1 and stays on.

```
$ ltlf-ek-synth --dfa-product \  
  --formula="X[!] k" \  
  --part-file part-a-k-o.txt \  
  --known-input-transducer tin-eventual-k.hoa
```

```
HOA: v1  
States: 2  
...  
exit code 0          (a controller)
```

$\mathcal{T}_{in} = \text{tin-delay-k}$ :  $k$  tracks the environment's  $a$ , one step behind.

```
$ ltlf-ek-synth --dfa-product \  
  --formula="X[!] k" \  
  --part-file part-a-k-o.txt \  
  --known-input-transducer tin-delay-k.hoa
```

```
UNREALIZABLE  
exit code 20
```

The verdict is a property of the *knowledge*, not of the formula. Nothing the controller can play changes either outcome —  $k$  is not its variable.

# The output is a checkable artifact

Realizable does not just print REALIZABLE — it prints the controller, as a Spot automaton:

```
HOA: v1
States: 2
Start: 0
AP: 2 "a" "o"
acc-name: all
properties: trans-labels explicit-labels state-acc complete deterministic
spot-state-player: 0 1
controllable-AP: 1
--BODY--
State: 0      [t] 1
State: 1      [t] 0
--END--
```

... and the same binary will check one for you:

```
$ ltlf-ek-synth --dfa-product --formula="X[!] k" --part-file part-a-k-o.txt \
  --known-input-transducer tin-eventual-k.hoa --model-check
SAFE
```

# Producing knowledge, not just consuming it

ltlf-ek-deps asks which variables are *forced* by the history — and can emit the answer as a transducer the methods then consume.

```
$ ltlf-ek-deps --formula="F(a ^ b)" --inputs a,b --outputs o --direction in
dependent inputs: a    (of a, b)
```

```
$ ltlf-ek-deps --formula="F(a ^ b)" --inputs a,b --outputs o --direction out
dependent outputs: none
```

- $X_{dep}$  **non-empty**: the environment's  $a$  is forced — the symmetric question, run against  $\neg\varphi$ .
- $Y_{dep}$  **correctly empty**: the system can always violate an output conjunct unconditionally, so nothing on the output side is ever forced here. Structural, not a gap in the tool.

## Early results

---

# What was measured

## Four parametric families, each scaling in $n$ :

- `cons-prunes` — the consistency filter *cons* genuinely cuts the product
- `cons-inert` — *cons* cuts nothing
- `mirror-small`, `mirror-degenerate`

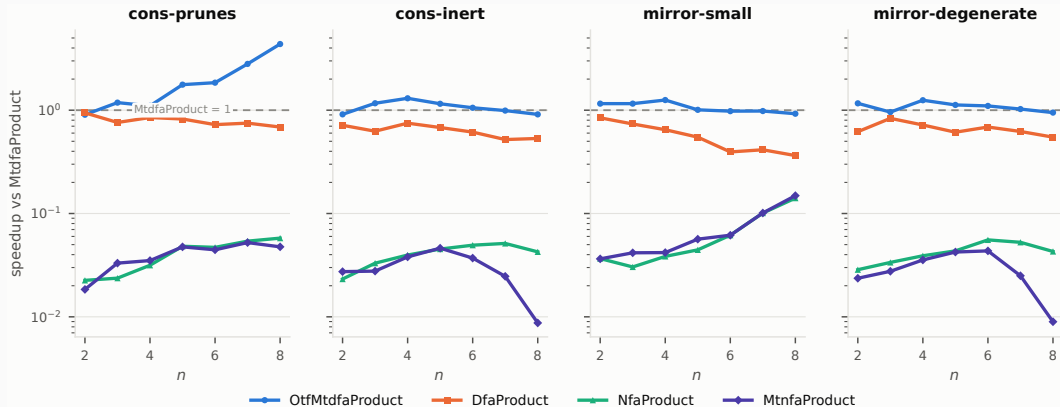
A fifth family, `parity-t3`, is excluded: it sits in an expressibility tier the comparison bars, and its declared realizability is wrong.

## Provenance

|         |                              |
|---------|------------------------------|
| build   | Release                      |
| Spot    | 2.15.1                       |
| $n$     | 2 ... 8, all pairs reached 8 |
| repeats | 3, 20 s per-case timeout     |
| sweep   | no case stopped early        |

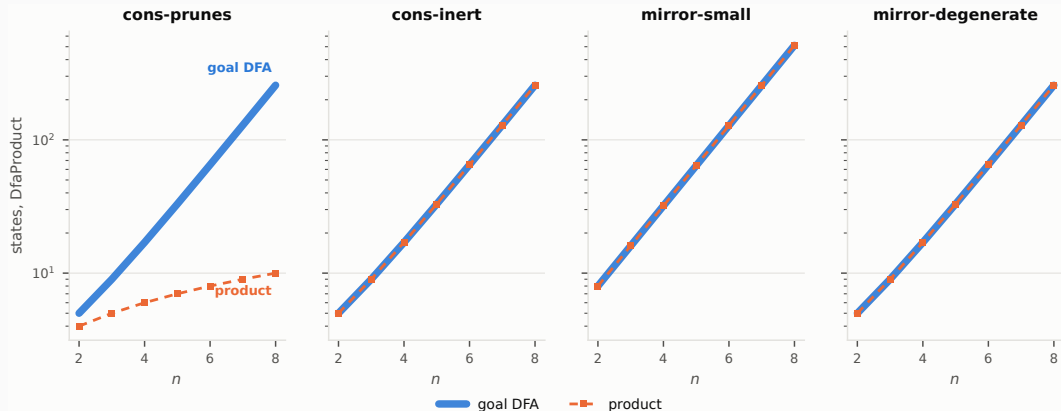
All five methods run every case; ratios are taken against `MtdfaProduct`.

# Speedup relative to MtdfaProduct



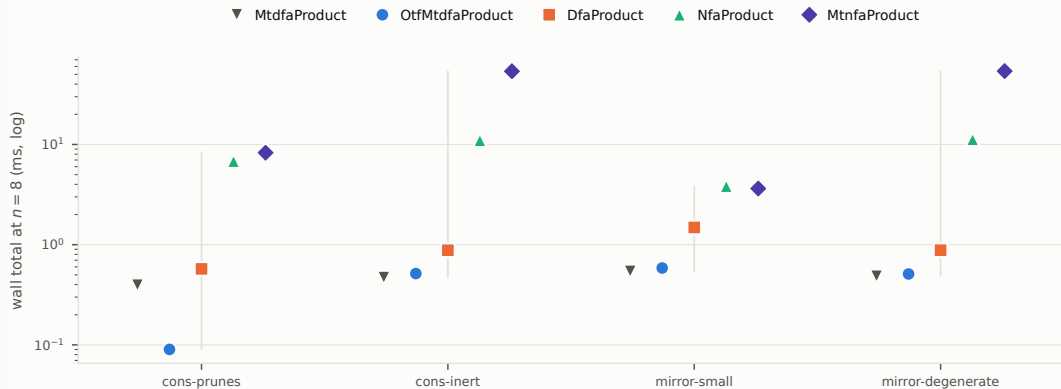
**One clean signal:** OtfMtdfaProduct pulls away ( $4.4\times$  at  $n=8$ ) exactly on cons-prunes — and costs essentially nothing ( $0.91\text{--}0.95\times$ ) on the three families where it does not help.

# Why: the goal blows up, the product does not



Sizes, not times — so this panel is build-independent. On **cons-prunes** the goal DFA grows exponentially while the product stays linear: that gap is the thing the on-the-fly method is exploiting. Where the two curves coincide, it has nothing to exploit.

## Absolute cost at $n = 8$



The MONA-backed methods sit one to two orders of magnitude higher throughout. Read that as the cost of shelling out to an external tool, *not* as a result about the NFA representation.

# 0 verdict mismatches

over **56** comparable cases

- Every case in the four families is re-encoded and raced against `ltlfsynt`; both verdicts are recorded per case.
- 14 further cases are *skipped by construction*, not silently passed — their tier makes the monolithic encoding illegitimate.

A cross-check on the verdicts, not on the timings — and it does not settle the reduction itself (appendix).

## Where this leaves us

**The prototype:** five interchangeable methods, two tools, controllers that can be emitted and checked, and a benchmark suite that reruns itself.

**The early results:**

- On-the-fly product is **roughly free when the consistency filter is inert** and  **$4.4\times$  when it prunes** — and the structural data says why.
- No verdict disagreement with `ltlfsynt` anywhere the comparison is legitimate.

**What these numbers are not:** four synthetic families,  $n \leq 8$ , one machine. A signal about where to look, not a cost model.

**Open:** the monolithic reduction (appendix), and the expressibility tier that currently bars a family from comparison.

## Appendix — the O5 boundary

On a *partial* transducer our verdict and `ltlfsynt`'s verdict on the monolithic encoding **diverge**. Partiality deletes letters for both players, and the two encodings do not delete the same ones.

- This is an **open theory question**, not a bug in either tool.
- It is why the benchmark race is restricted to the tier where the monolithic encoding is known legitimate.
- The reduction is a **conjecture**; nothing here proves it.

Witness and full discussion: `docs/runs/2026-08-09-acceptance-mark-edgeless.md`, and `tools-and-capabilities.md` §4f.

## Appendix — why the build type mattered

The same sweep, Debug vs Release, speedup of `OtfMtdfaProduct` at  $n = 8$ :

| family            | Debug | Release     |
|-------------------|-------|-------------|
| cons-prunes       | 3.09  | <b>4.37</b> |
| cons-inert        | 0.39  | <b>0.91</b> |
| mirror-small      | 0.42  | <b>0.92</b> |
| mirror-degenerate | 0.40  | <b>0.95</b> |

Debug materially understated the method: it read as “wins  $3\times$  here, costs you  $2.5\times$  everywhere else”. The methods do not pay equally for assertions, so cross-method ratios are not build-portable.

Every figure in this deck is from the Release sweep, `docs/runs/2026-08-11-benchmarks-release.xlsx`.